Assignment 01 – Music DJ

Submission Deadline: Sunday 01 March 2026

Problem: Write a C++ program for processing voice data read from (.wav) files.

1. **Allocate Memory:**
   Write a function that allocates memory for a dynamic array of any given size.
   ```
   void allocateArray(unsigned char *& arr, int size);
   ```

2. **Deallocate Memory:**
   Write a function to deallocate the Array.
   All memory must be deallocated properly at end of program.

   ```
   void deallocateArray(unsigned char *& arr);
   ```

3. **Print:**
   Write a function to print the contents of array of any given size.

4. **Doubles an Array:**
   Write a function that doubles an array by duplicating items in the array and returns doubled array. This function will not change the original array , make careful selection of function parameters.

   For Example:
   Original Array: `{4,2,1,9,1,3,6}`
   Doubled Array: `{4,4,2,2,1,1,9,9,1,1,3,3,6,6}`

   The above is a special case of a procedure called up sampling, where you are increasing the number of samples in a signal.

5. **Shrink an Array:**
   Write a function that reduces an array's size to half by deleting every other item of the array and returns shirked array . This function will not change the original array, make careful selection of function parameters.

   For Example:
   Original Array: `{4,2,1,9,1,3,6}`
   Halved Array: `{ 4,1,1,6}`

   The above is a special case of a procedure called down sampling of a signal, which takes a selected set of values from the original array and reduces the samples to half.

6. **Fill with Mean:**
   Write a function **`FillWithMean`** that takes two arrays, **`in`** and **`out`** , and a positive number **`N`** as parameters.

The function then fills each index **i** of the array **out** with the mean/ average of **N*2 +1** value in the neighborhood of index **i** from the array **in**. That is, it will take the mean of the **i**th number and **N** numbers to its left and **N** numbers to its right. You also need to take care of the boundaries as shown in the examples below. The input and output arrays should be different. This procedure is called the method of moving averages.

EXAMPLE 1 with N = 1

index	0	1	2	3	4	5
in	4	5	3	6	1	3
out	Mean(4,5) = 9/2	Mean(4,5,3) = 4	Mean(5,3,6) = 14/3	Mean(3,6,1) = 10/3	Mean(6,1,3) = 10/3	Mean(1,3) = 2

EXAMPLE 2 with N = 2

index	0	1	2	3	4	5
in	4	5	3	6	1	3
out	Mean(4,5,3) = 4	Mean(4,5,3,6) = 18/4	Mean(4,5,3,6,1) = 19/5	Mean(5,3,6,1,3) = 18/5	Mean(3,6,1,3) = 13/4	Mean(6,1,3) = 10/3

About Audio Wav Files

Wav files are nothing but sequences of arrays, containing sound samples. The most basic audio files are 'mono', which means only one channel with 8000 samples in one second, each sample being one byte long. You are provided with two basic audio files reading and writing functions.

The **readWavFile** function will read the sound values in an array and return the sampling rate (bytes per second). Increasing the sampling rate of audio will improve the sound quality. Decreasing the sampling rate will decrease the sound quality. The wav file functions provided will give you a basic idea about audio storage and manipulation.

```
readWavFile("sallimono.wav", arr, size, samplingRate);
//will read the audio file
writeWavFile("sallimono.wav", arrarr, sizesize, samplingRate);
//will write the audio file
```

Notice:

A. The data is read into an array of type **unsigned char**, which means each value of audio file is one byte long an unsigned number.
B. The function **readWavFile** will also place the bytes read in the variable size . Size is an input output parameter.
    a. If the input size is more than audio size, then size is changed to actual audio size.
    b. If size is less than the audio file's size, then only the length number of bytes are read in the array.
C. Th e function also returns the sampling rate, which indicates how many audio values were collected in one second. You can print this value to see what it is.
D. You can also print the array to see the values stored in it.

7. **Read from File:**
   Write a function **`readFromFile`** which takes file name from user as input and then calls **`readWavFile`** function to read the contents of the array from the file.

8. **Up Sample an Audio File:**
   Write a function **`upSampleAudio`** which up samples already read sound file by calling double array function and then write data back in a new wav file by calling function **`writeWavFile`** function. After writing the new array, listen to the new audio file and see what your observations are.
   Notice how the pitch is changed!

9. **Down Sample an Audio File:**
   Write a function **`downSampleAudio`** which down samples already read wav file by calling shrink array function and then write data back in the new wav file by calling function **`writeWavFile`**. After writing the new array, listen to the new audio file and see what your observations are.

10. **Apply Moving Average Filter to the Audio Array:**
    Write a function **`movingAverageFilter`** which will create the copy of original wave file and will apply moving average filter by calling fill with mean function and then write data back in wav file by calling function **`writeWavFile`**. Listen to the new audio file and note your observations. Repeat this for different values of **`N`** like, **`1,3, 5, 15, 100`**.

11. **Mix Two Audio Signals Together:**
    Write a function called **`mergeArray`**, which takes as input two arrays and their sizes and returns a third array merged with values of first two arrays. For example:

    Array1: **`{1,2,3,4,5,6,7,8}`**
    Array2: **`{10,29,50}`**
    Mergedarray: **`{1,10,2,29,2,50,4,5,6,7,8}`**

    Read **`dhani.wav`** and **`sallimono.wav`** and construct a third array which has mixed data of both arrays (you need to call **`mergeArray`**). Write the resulting in a wav file and listen to the wav file. When saving the wav file, save its two copies one with salli's sampling rate and other with dhani's sampling rate . What are your observations after listening?

12. **Menu:**
    For all the parts given above make a menu function to access and test each function.
    You can repeat this for other music files (.wav) also.

Submission Instructions

1. Use meaningful variable names
2. YOU MUST DEALLOCATE ALL MEMORY PROPERLY, YOUR CODE SHOULD NOT HAVE ANY MEMORY LEAKS OR DANGLING POINTERS.
3. Indent your program so that statements inside a block can be distinguished from another block
4. Use const type instead of using hard coded numbers
5. You ARE NOT ALLOWED to use break, continue and goto
6. You ARE NOT ALLOWED to use global variables
7. You ARE NOT ALLOWED to use more than one return statement in a function
8. All functions that compute values should have NO cin and cout. Exchange all data via parameters.
9. Functions that compute values should be kept separate from those that draw on screen
10. Spaghetti code and confusing code is not acceptable
11. Comment your code properly